
IIC2523

Sistemas Distribuidos

— Hernán F. Valdivieso López —
(2026 - 2 / Clase 12)

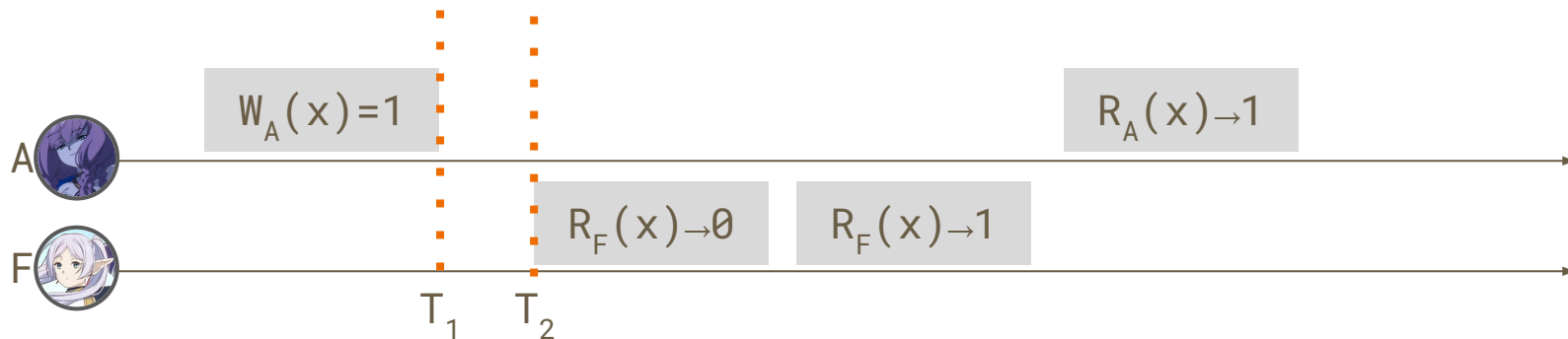
Replicación de Datos II

¿Cómo aseguramos consistencia en las réplicas?

Temas de la clase

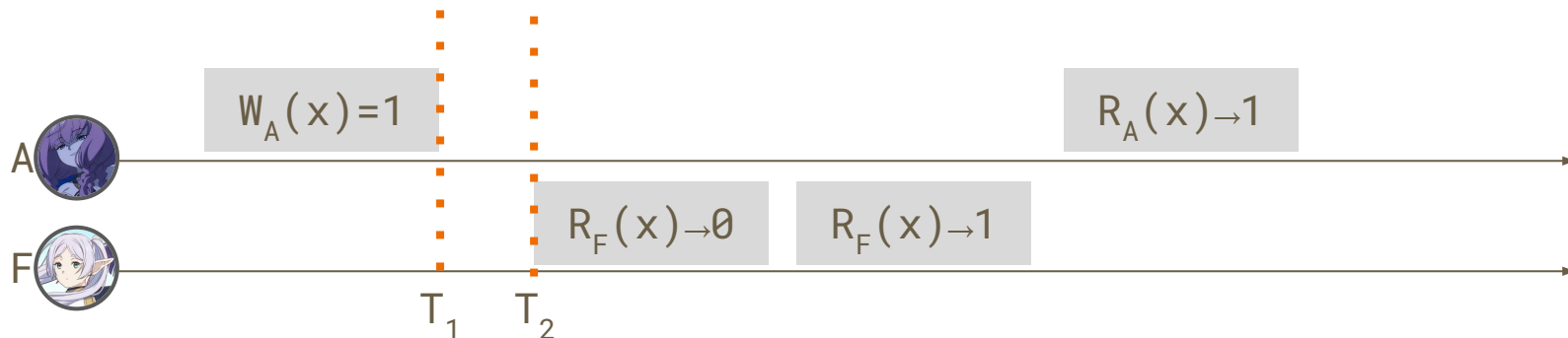
1. Clase pasada
 - a. Recordatorio y replicación basada quórum.
2. Modelos de consistencia Causal
3. Modelos de consistencia Eventual
 - a. ¿En qué consiste?
 - b. Protocolo *Gossip*

Clase Pasada - Consistencia Secuencial y Fuerte



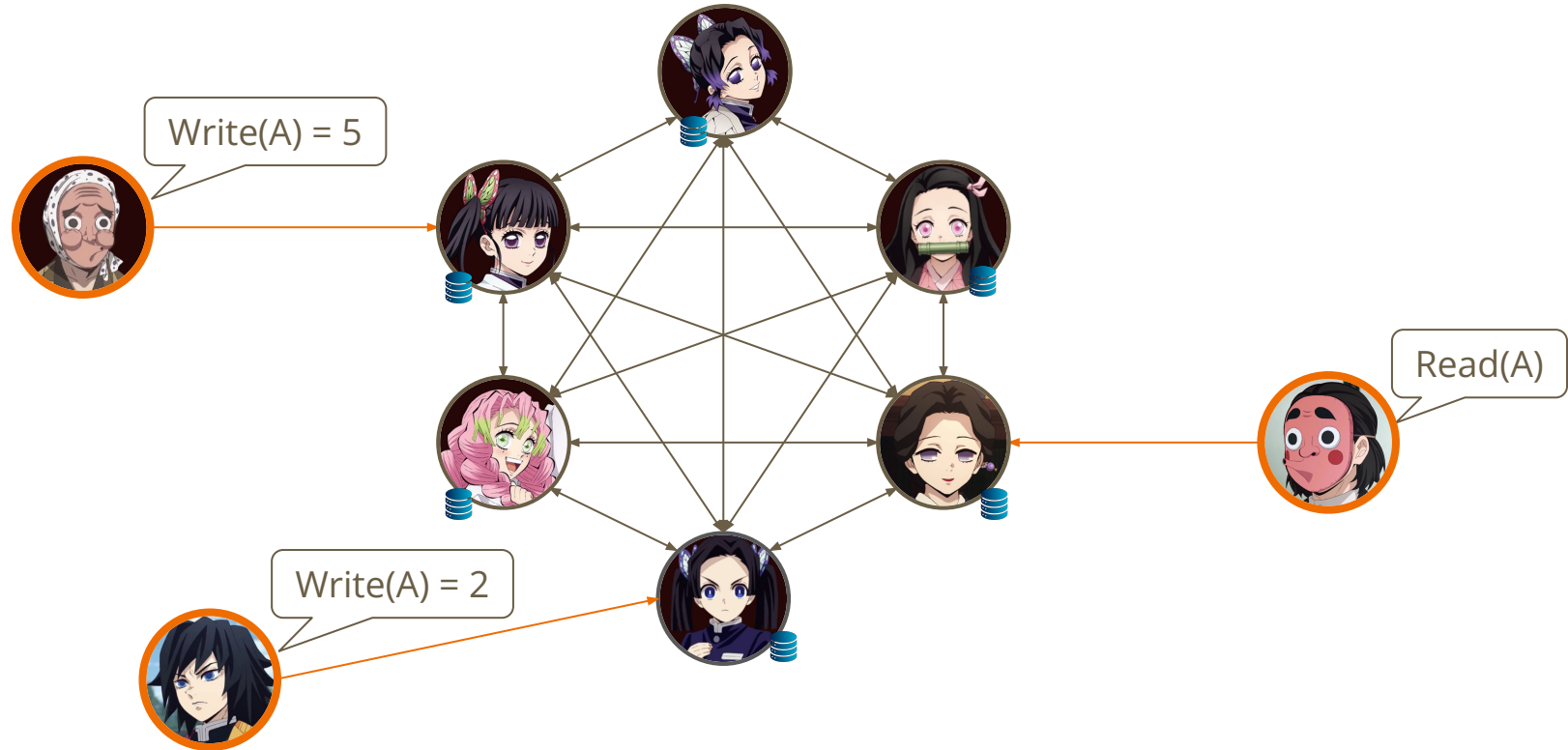
◆ Asumiendo $T_1 < T_2$ ¿Se cumple algún modelo de consistencia?

Clase Pasada - Consistencia Secuencial y Fuerte

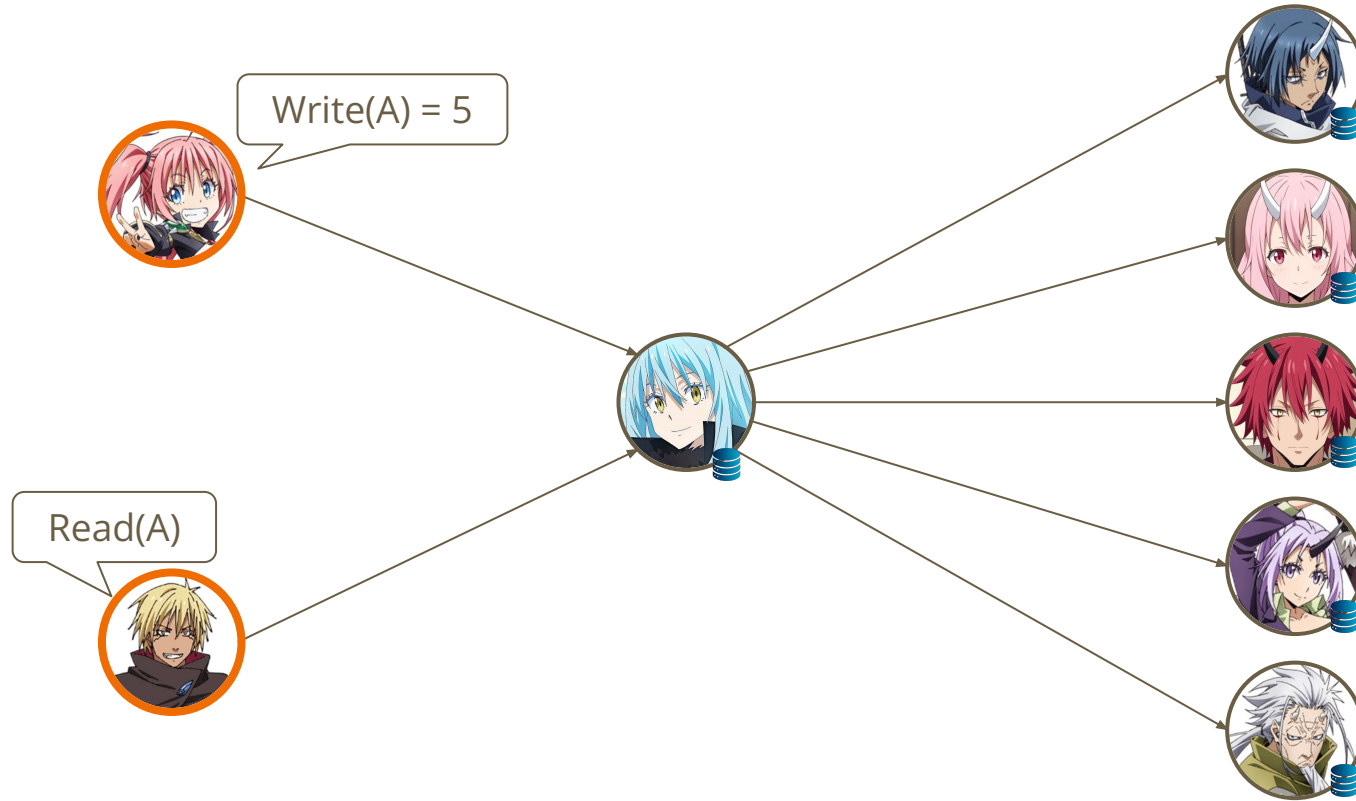


- ◆ Asumiendo $T_1 < T_2$ ¿Se cumple algún modelo de consistencia?
- ◆ Es consistente secuencialmente, pero no es fuerte.
- ◆ Puede ser que nodo F leyó la variable X antes que le llegara la propagación del cambio de X por el nodo A.
 - ◆ Existe una secuencia **válida secuencial de operaciones** ($R_F \rightarrow W_A \rightarrow R_F \rightarrow R_A$)
 - ◆ No respeta el requisito de **tiempo real** de la consistencia fuerte.

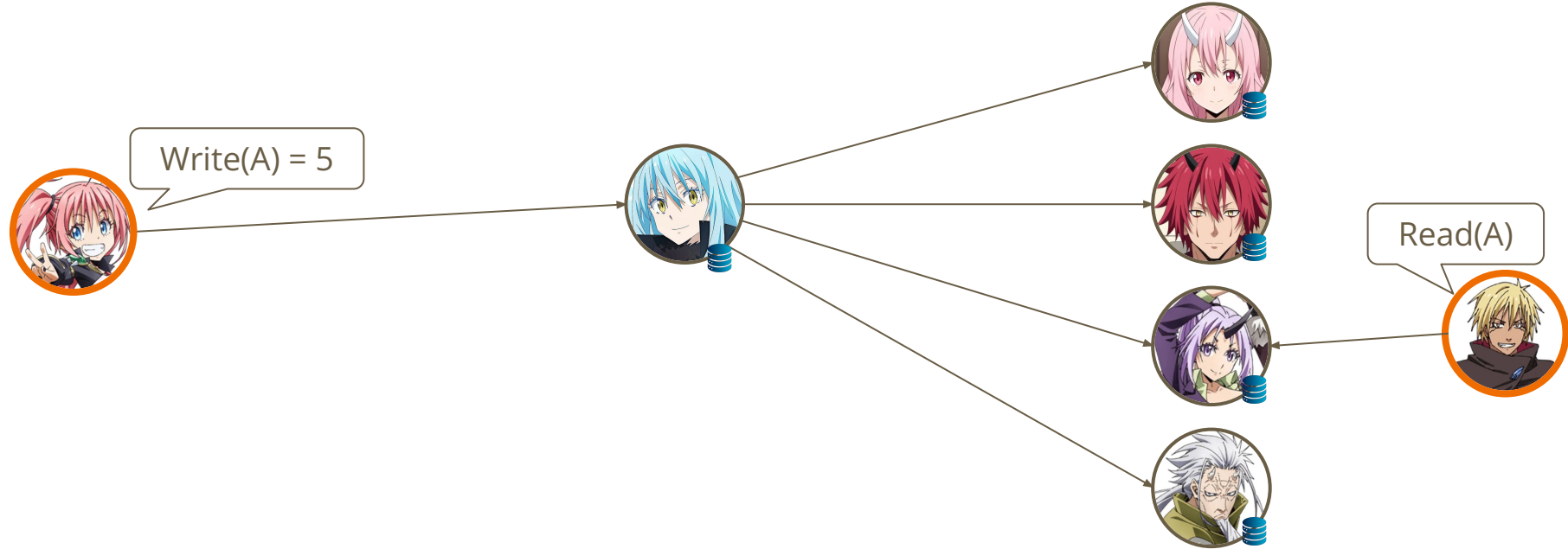
Protocolos de Replicación - *Replicated-Write Protocols*



Protocolos de Replicación - *Primary-Based Protocols*



Protocolos de Replicación - *Primary-Based Protocols*

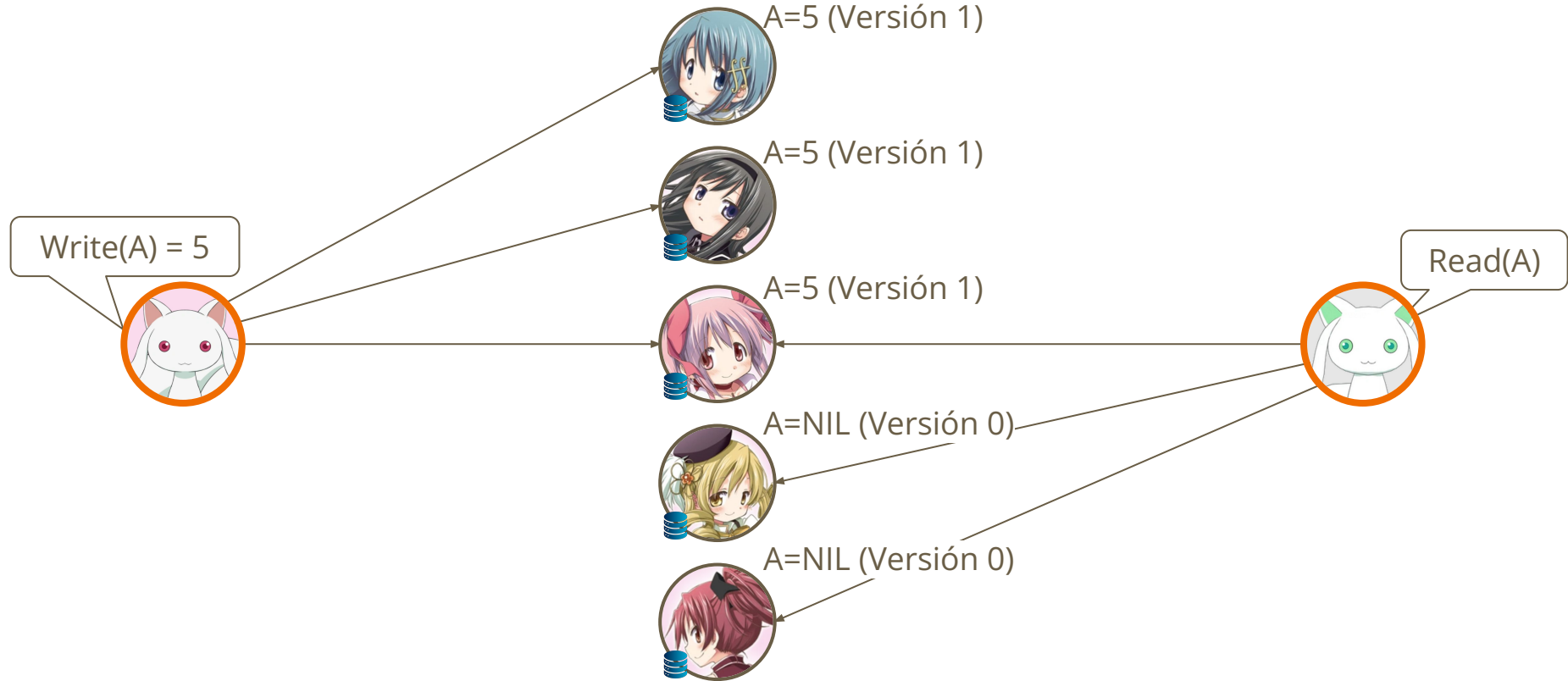


Protocolos de Replicación - *Primary-Based Protocols*



Protocolos de Replicación - *Quorum-Based*

Protocolos de Replicación - *Quorum-Based*



Protocolos de Replicación - *Quorum-Based*

- ◆ Para realizar operaciones de lectura o escritura, un subgrupo de réplicas debe participar y dar su "voto", formando un quórum.
- ◆ Reglas de Quórum
 - ◆ Sea **N** la cantidad de nodos, **RQ** el quórum de lectura y **WQ** el quórum de escritura.
 - ◆ Para obtener un dato, se debe consultar a RQ nodos y el dato más reciente es el elegido.
 - ◆ Para escribir un dato, se debe obtener previamente la aprobación de WQ nodos.
 - ◆ La consistencia se garantiza si la intersección de cualquier quórum de lectura y cualquier quórum de escritura es no vacía: $(RQ + WQ > N)$
 - ◆ Se exige $WQ > N/2$ para evitar conflictos de escrituras.
 - ◆ Para cualquier operación, si no se logra el quórum esperado, la operación no se ejecuta.

Protocolos de Replicación - *Quorum-Based*

Consistencia (asumiendo $RQ + WQ > N$)

- ◆ Previene inconsistencia al momento de obtener el dato.
 - ◆ Dado que al menos existirá 1 nodo que será parte de lectura y escritura. Así que siempre se tendrá el dato más reciente.
 - ◆ Pueden existir réplicas con datos que no se han actualizado con el cambio de un *write*.
- ◆ Se garantiza la **consistencia secuencial**.

Protocolos de Replicación - *Quorum-Based*

Ventajas (asumiendo $RQ + WQ > N$)

- ◆ Es muy robusto ante particiones de red, ya que solo la partición que logra formar un quórum puede continuar operando, previniendo inconsistencias.
- ◆ Mientras se respete el Quórum, tolera una alta cantidad de fallas de nodos.

Desventajas

- ◆ Mayor latencia de las operaciones
- ◆ La probabilidad de que no haya disponibilidad va a aumentar.

Protocolos de Replicación - *Quorum-Based*

Variaciones en Quórum

- ◆ *Read-One Write-All (ROWA)* $RQ = 1$ y $WQ = N$. Rápida lectura pero lenta escritura.
- ◆ *Majority Quorum (Voting-Based Quorum)*. Tanto RQ y WQ deben ser mayor a $N/2$.
- ◆ *Weighted Voting*. Cada nodo tiene un peso, y el quórum se define en función de la suma de pesos, en vez de la cantidad de nodos.
- ◆ *Dynamic Voting*. Ajusta el N , RQ y WQ según la cantidad de nodos activos.

Modelo de Consistencia Causal

Consistencia Causal [Huto & Ahamad, 1990]

- ◆ El modelo de consistencia causal es un debilitamiento de la consistencia secuencial.
- ◆ Hace una distinción entre los eventos que están potencialmente relacionados causalmente y aquellos que no lo están.

Consistencia Causal [Huto & Ahamad, 1990]

- ◆ El modelo de consistencia causal es un debilitamiento de la consistencia secuencial.
- ◆ Hace una distinción entre los eventos que están potencialmente relacionados causalmente y aquellos que no lo están.
 - ◆ Si un evento B es causado o influenciado por un evento anterior A, la causalidad requiere que todos los demás vean primero A y luego B.
 - ◆ Si un evento C, ejecutado por otro nodo de forma concurrente a B, no tiene ninguna causa, puede estar posicionado en diferentes partes para diversas réplicas. No está "obligado" a una única secuencia.

Consistencia Causal [Huto & Ahamad, 1990]

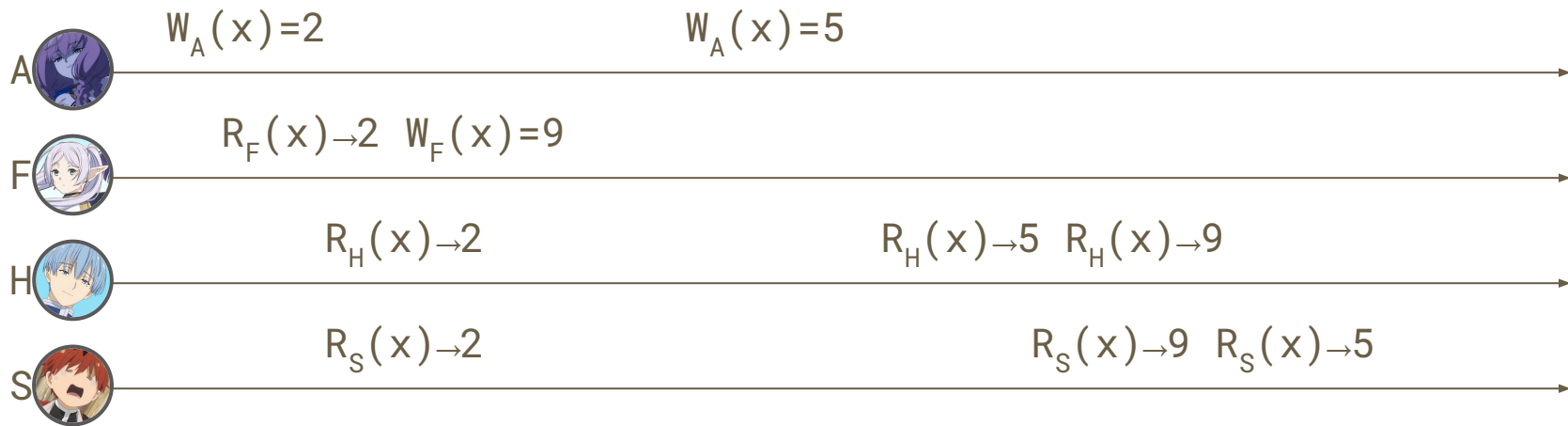
Ejemplo foro de discusión

- ◆ Persona 1 comenta una publicación de política.
- ◆ Persona 2 lee el comentario de Persona 1 y escribe una funa dicha persona.
- ◆ Persona 3 publica un *post* sobre Anime.

En un sistema con consistencia causal ocurre que:

- ◆ Todos los usuarios ven el comentario de la persona 1 antes de la funa hecha por la persona 2.
- ◆ No se impone un orden global entre el *post* de "Politica" y "Anime", ya que no hay una relación causal directa entre ellos.

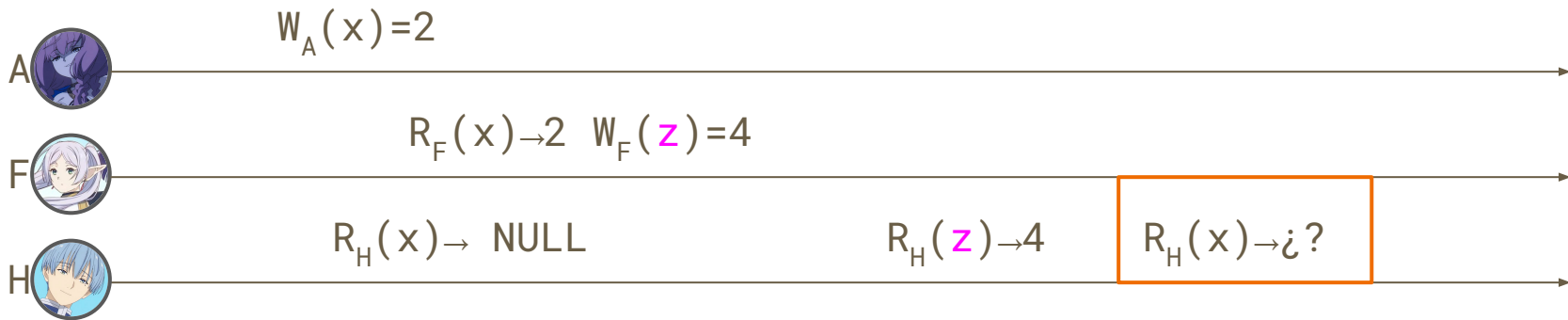
Consistencia Causal [Huto & Ahamad, 1990]



- ◆ Nodos H y S ven los *write* en órdenes distintos $\rightarrow$ No es consistente secuencial
- ◆ Nodo F leyó la escritura del nodo A ($x=2$) antes de escribir él ($x=9$)
 - ◆ Solo se exige que se respete que 2 viene antes de 9. Si algún nodo lee el 2 posterior al 9, se rompe la consistencia causal.

Consistencia Causal [Huto & Ahamad, 1990]

- ◆ Solo se exige causalidad.



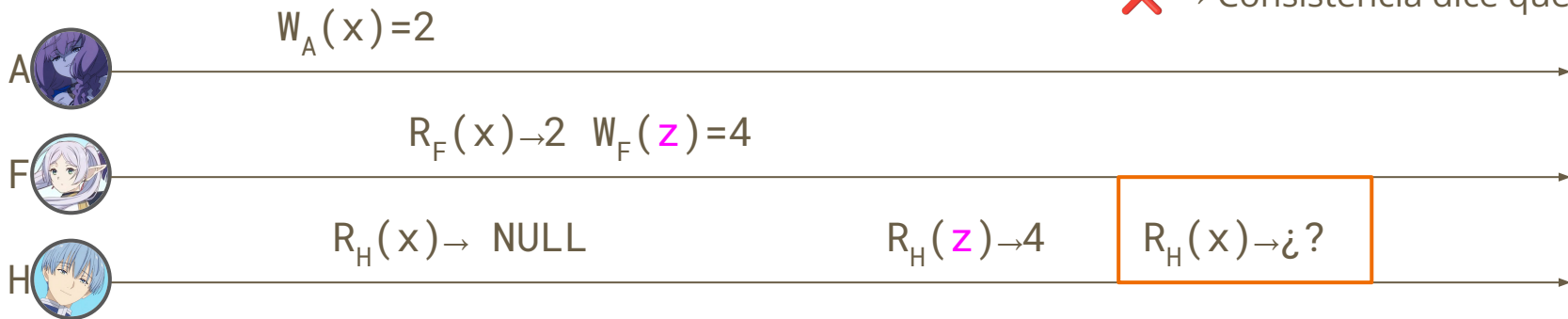
- ◆ ¿Qué podemos decir del valor de **X** al final si aseguramos consistencia causal?

Consistencia Causal [Huto & Ahamad, 1990]

- ◆ Solo se exige causalidad.

✓ → Consistencia dice que **Si**

✗ → Consistencia dice que **No**



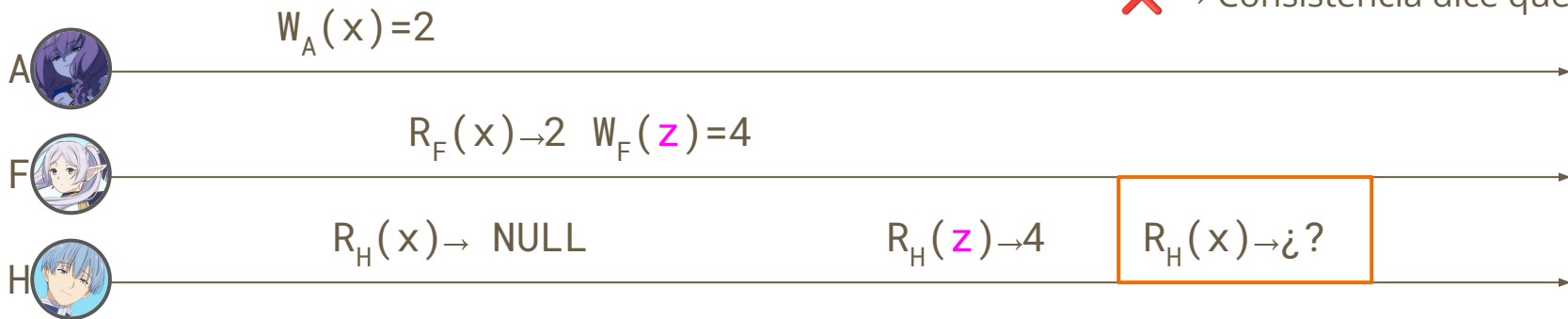
- ◆ ¿Qué podemos decir del valor de **X** al final si aseguramos consistencia causal?
 - ◆ ¿Puede tomar el valor 2?
 - ◆ ¿Puede tomar el valor NULL (sin definir la variable todavía)?

Consistencia Causal [Huto & Ahamad, 1990]

- ◆ Solo se exige causalidad.

✓ → Consistencia dice que **Si**

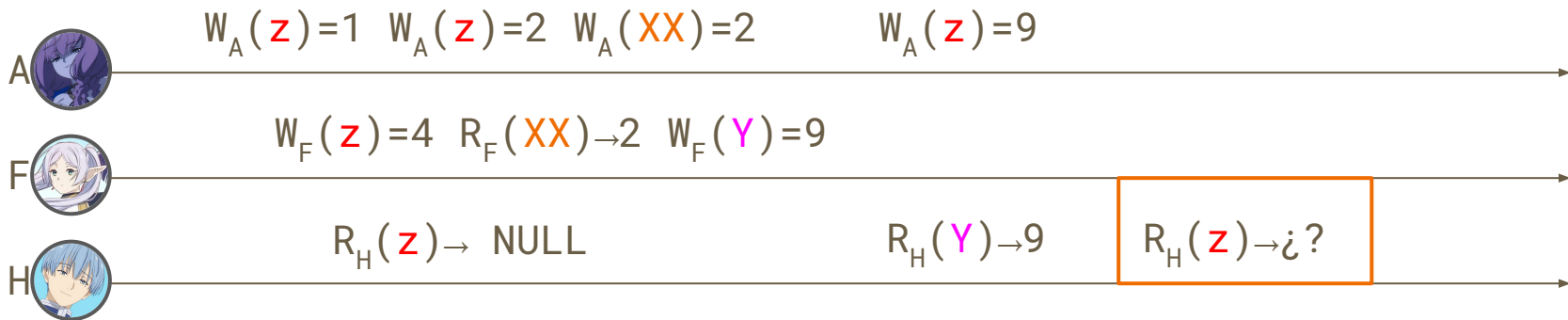
✗ → Consistencia dice que **No**



- ◆ ¿Qué podemos decir del valor de **X** al final si aseguramos consistencia causal?
 - ◆ ¿Puede tomar el valor 2? ✓
 - ◆ ¿Puede tomar el valor NULL (sin definir la variable todavía)? ✗

Consistencia Causal [Huto & Ahamad, 1990]

◆ Solo se exige causalidad.



◆ ¿Qué podemos decir del valor de $\mathbf{Z}$ al final si aseguramos consistencia causal?

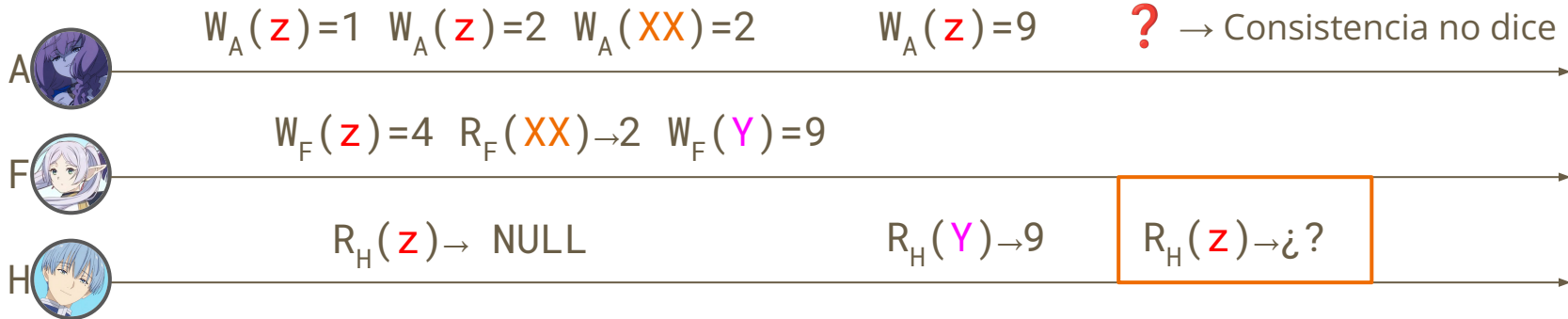
Consistencia Causal [Huto & Ahamad, 1990]

◆ Solo se exige causalidad.

✓ → Consistencia dice que **Si**

✗ → Consistencia dice que **No**

? → Consistencia no dice nada.



◆ ¿Qué podemos decir del valor de **Z** al final si aseguramos consistencia causal?

- ◆ ¿Puede tomar el valor 1? ○ ¿Puede tomar el valor 9?
- ◆ ¿Puede tomar el valor 2? ○ ¿Puede tomar el valor NULL?
- ◆ ¿Puede tomar el valor 4?

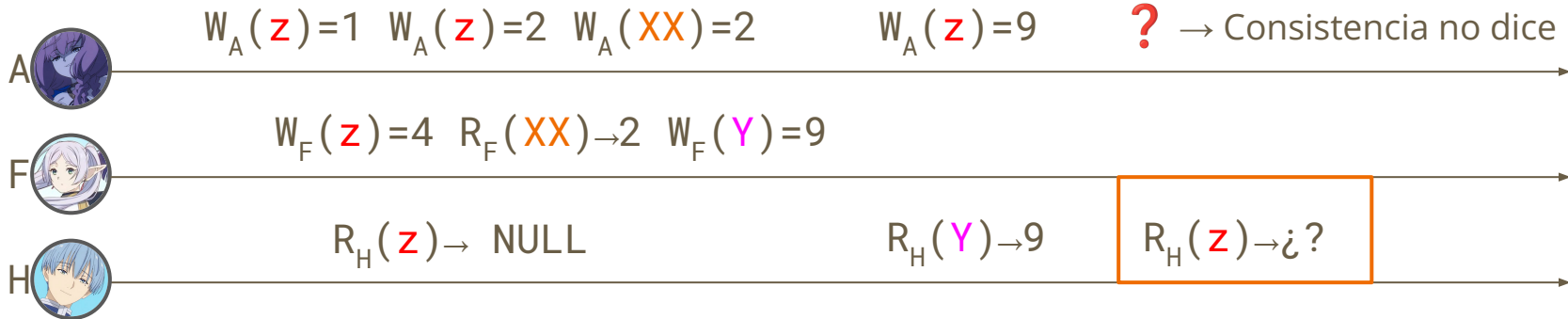
Consistencia Causal [Huto & Ahamad, 1990]

◆ Solo se exige causalidad.

✓ → Consistencia dice que **Si**

✗ → Consistencia dice que **No**

? → Consistencia no dice nada.



◆ ¿Qué podemos decir del valor de Z al final si aseguramos consistencia causal?

- ◆ ¿Puede tomar el valor 1? ✗
- ◆ ¿Puede tomar el valor 2? ✓
- ◆ ¿Puede tomar el valor 4? ✓
- ¿Puede tomar el valor 9? ?
- ¿Puede tomar el valor NULL? ✗

Modelo de Consistencia Eventual

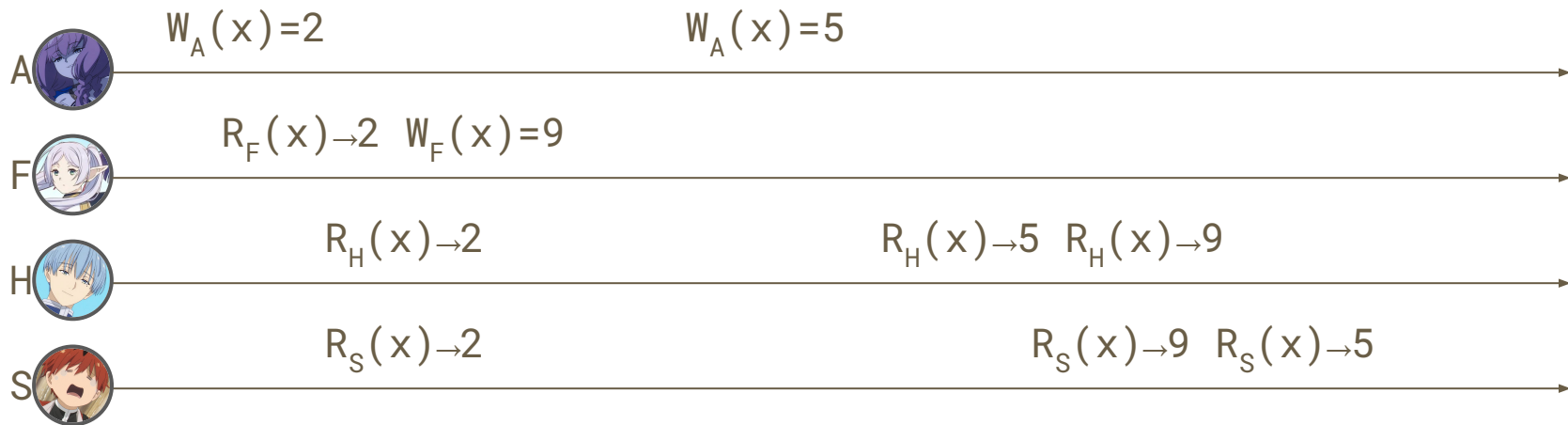
¿En qué consiste?

Protocolo *Gossip*

Consistencia Eventual [Vogels, 2009]

- ◆ Modelo que prioriza la alta disponibilidad y el rendimiento sobre la consistencia.
- ◆ Si 2 o más nodos no están escribiendo exactamente al mismo tiempo (no hay conflicto de escritura), las réplicas convergerá hacia copias idénticas entre sí.
- ◆ Se garantiza que las actualizaciones se propagan eventualmente a las réplicas.
 - ◆ Esto permite una relajación en la consistencia entre réplicas. Dos clientes pueden observar réplicas diferentes, e incluso un cliente puede observar datos obsoletos.
- ◆ En caso que exista un conflicto de escritura, necesitamos alguna estrategia... por ejemplo, se escoge uno que sobrescriba al otro.

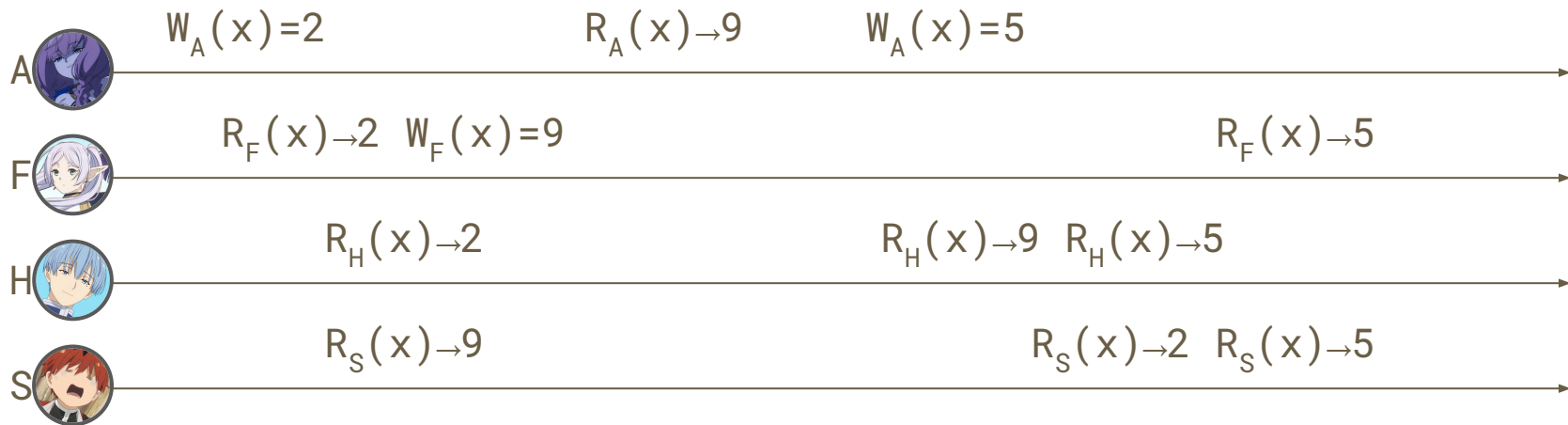
Consistencia Eventual [Vogels, 2009]



◆ Vimos anteriormente que este caso es consistente causal, pero no es consistente eventual.

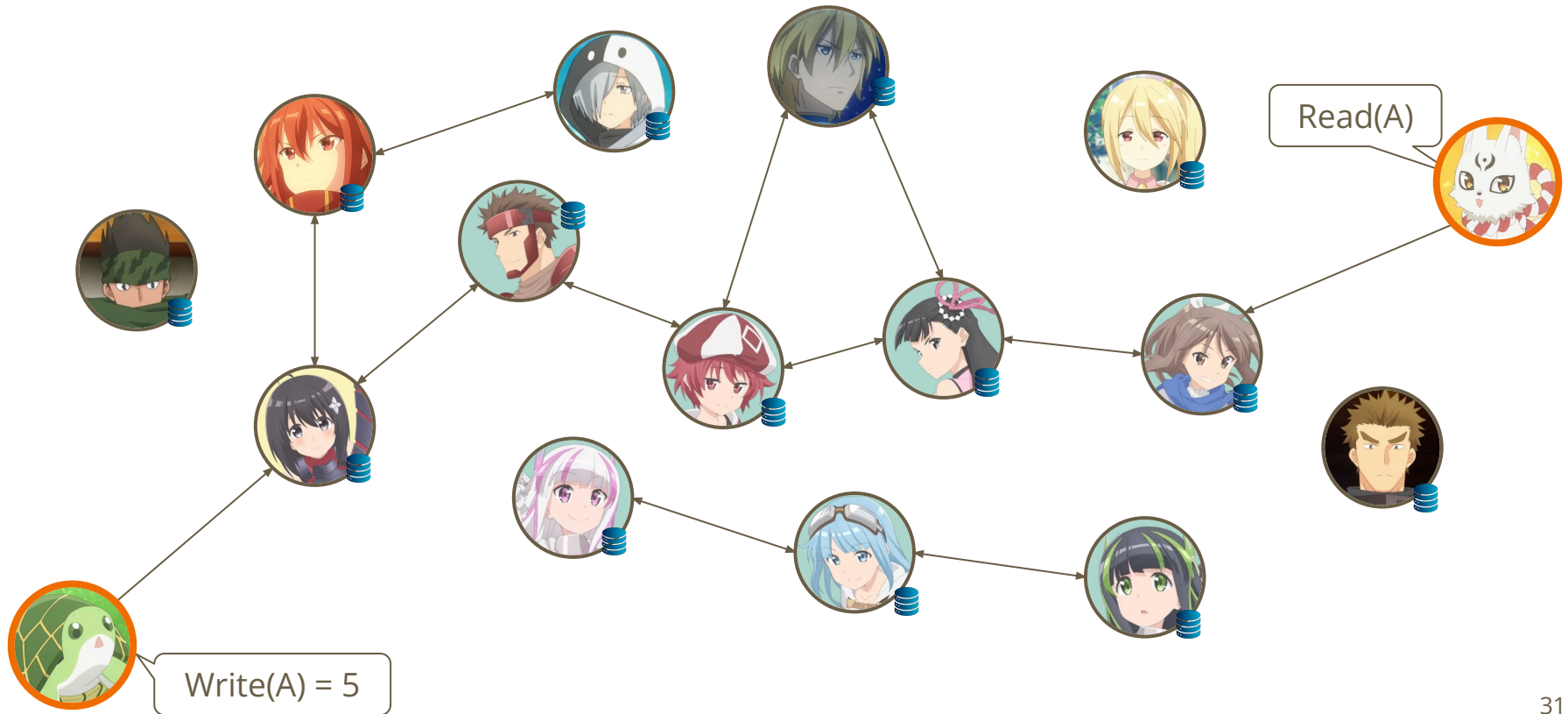
- ◆ H primero recibió $W_A(x)=5$ y luego recibió $W_F(x)=9$
- ◆ S primero recibió $W_F(x)=9$ y luego recibió $W_A(x)=5$
- ◆ No van a llegar a un resultado igual.

Consistencia Eventual [Vogels, 2009]



- ◆ Nodos H y S ven los *write* en distinto orden $\rightarrow$ No es consistente secuencial.
- ◆ Nodo S ve primero que x es 9 y luego que es 2 $\rightarrow$ No es consistente causal
 - ◆ Ojo que este caso sigue siendo factible... Si la comunicación de $A \rightarrow S$ es más lenta, puede pasar que $W_A(x)=2$ llegue después que nodo F propagó su cambio.
- ◆ Todos llegan finalmente a ver $x=5 \rightarrow$ Consistente eventual

Protocolos de Replicación - *Gossip*



Protocolos de Replicación - *Gossip*

- ◆ También conocido como Protocolo Epidémico.
 - ◆ Un nodo "infectado" con nuevos datos los propaga a sus vecinos directos, y ellos a su vez lo propagan a los demás.
- ◆ Un enfoque *lazy* para la propagación de actualizaciones. Las réplicas intercambian información sobre sus actualizaciones de estado más recientes de forma periódica con un subconjunto de sus vecinos directos.

Protocolos de Replicación - *Gossip*

Consistencia

- ◆ Se espera que todas las réplicas eventualmente tengan la misma información, **pero no se garantiza el orden en que observarán las actualizaciones.**
 - ◆ Consistencia eventual.

Protocolos de Replicación - *Gossip*

Consistencia

- ◆ Se espera que todas las réplicas eventualmente tengan la misma información, **pero no se garantiza el orden en que observarán las actualizaciones.**
 - ◆ Consistencia eventual.
- ◆ ¿Se puede mejorar su consistencia?

Protocolos de Replicación - *Gossip*

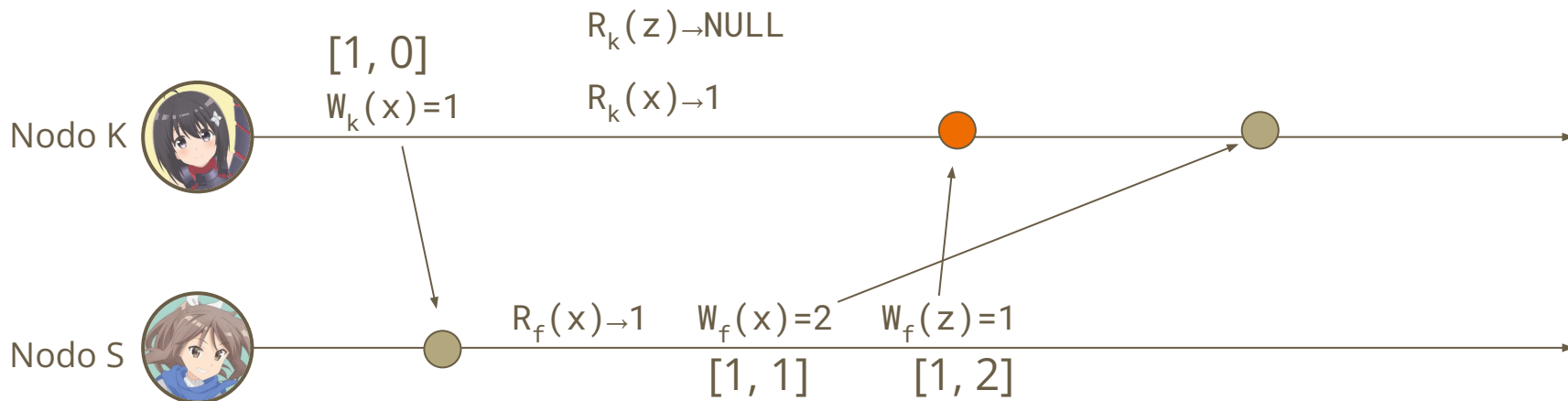
Consistencia

- ◆ Se espera que todas las réplicas eventualmente tengan la misma información, **pero no se garantiza el orden en que observarán las actualizaciones.**
 - ◆ Consistencia eventual.
- ◆ ¿Se puede mejorar su consistencia? Sí. Podemos incorporar:
 - ◆ **Vectores lógicos:** registran qué actualizaciones (*write*) conoce/aplicó cada réplica.
 - ◆ **Regla de actualización:** una réplica no aplica una actualización mientras alguna de sus dependencias causales no haya sido aplicada.
 - ◆ De esta forma, se puede garantizar **consistencia causal**.

Protocolos de Replicación - *Gossip*

Consistencia con vectores lógicos + regla de actualización

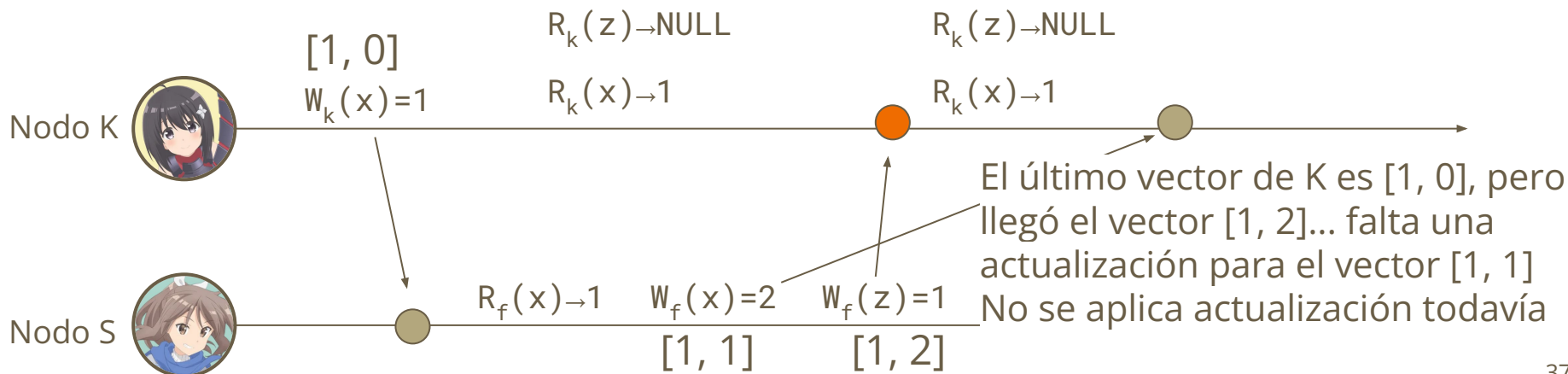
- ◆ **Vectores lógicos:** registran qué actualizaciones (*write*) conoce/aplicó cada réplica.
- ◆ **Regla de actualización:** una réplica no aplica una actualización mientras alguna de sus dependencias causales no haya sido aplicada.



Protocolos de Replicación - *Gossip*

Consistencia con vectores lógicos + regla de actualización

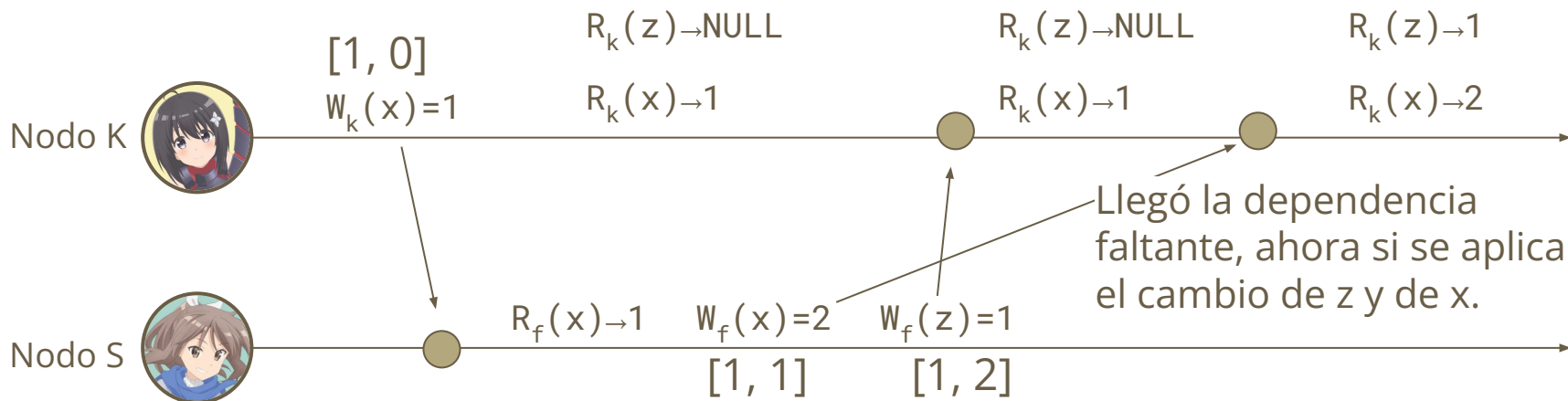
- ◆ **Vectores lógicos:** registran qué actualizaciones (*write*) conoce/aplicó cada réplica.
- ◆ **Regla de actualización:** una réplica no aplica una actualización mientras alguna de sus dependencias causales no haya sido aplicada.



Protocolos de Replicación - *Gossip*

Consistencia con vectores lógicos + regla de actualización

- ◆ **Vectores lógicos:** registran qué actualizaciones (*write*) conoce/aplicó cada réplica.
- ◆ **Regla de actualización:** una réplica no aplica una actualización mientras alguna de sus dependencias causales no haya sido aplicada.



Protocolos de Replicación - *Gossip*

Ventajas

- ◆ Extremadamente robusto, tolerante a fallas de nodos y escalables para sistemas muy grandes y dinámicos.
- ◆ No requieren un coordinador central.
- ◆ Muy directo de implementar.

Desventajas

- ◆ No son adecuados para aplicaciones que requieren consistencia estricta e inmediata, por ejemplo, transacciones financieras.
- ◆ La consistencia eventual puede tomar más tiempo de lo que uno desea.
- ◆ Alta redundancia de mensajes.

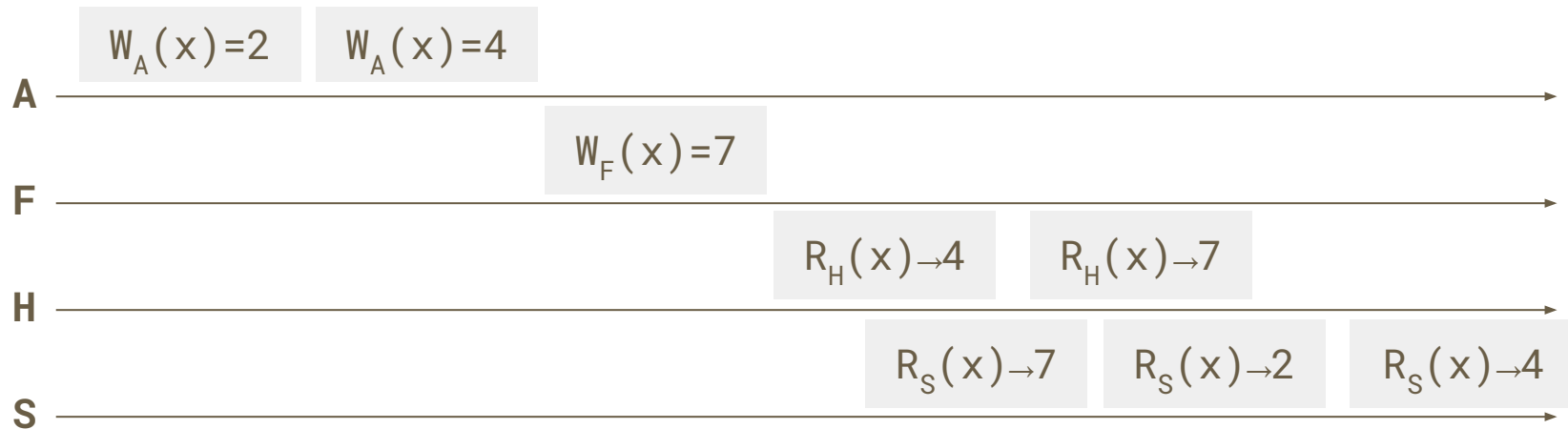
Protocolos de Replicación

¿Cómo se ven reflejados en la actualidad?

- ◆ [Apache Cassandra](#) (Base de datos NoSQL) utiliza el protocolo *Gossip* para la comunicación entre nodos y el protocolo basado en *Quorum* para las réplicas de datos.
- ◆ [Amazon Dynamo](#) (Base de datos NoSQL de uso interno de Amazon) utiliza el protocolo *Gossip* para trackear el estado del sistema y una variación del protocolo basado en *Quorum* para la consistencia de las réplicas.
- ◆ [Consul](#) (administrador de red privada para servicios distribuidos) utiliza una variación del protocolo *gossip* ([SWIM](#)) para administrar los servicios de la red y comunicar los mensajes.
- ◆ [Google Spanner](#) (servicio de base de datos distribuido) utiliza *Quorum* para la escritura de datos.

Poniendo a prueba lo que hemos aprendido

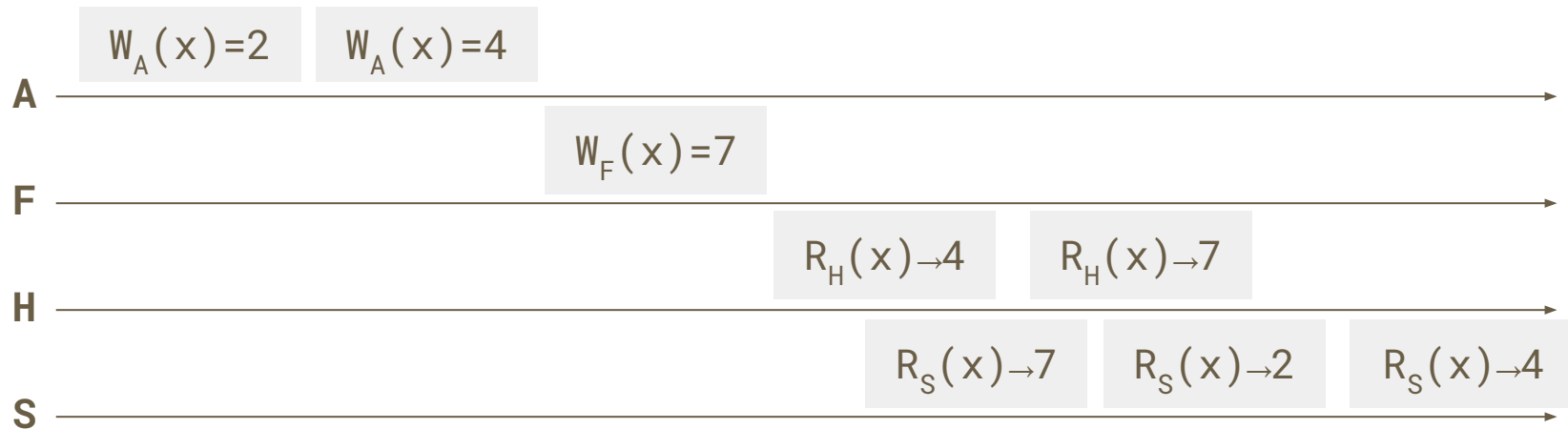
Para cada modelo indique si se cumple o no



- Consistencia Secuencial [1979]
- Consistencia Fuerte [1986]
- Consistencia Causal [1990]
- Consistencia Eventual [2009]

Poniendo a prueba lo que hemos aprendido

Para cada modelo indique si se cumple o no



- a. Consistencia Secuencial [1979] 
- b. Consistencia Fuerte [1986] 
- c. Consistencia Causal [1990] 
- d. Consistencia Eventual [2009] 

Próximos eventos

Próxima clase

- ◆ Transacciones Distribuidas
 - ◆ ¿Cómo administramos un conjunto de operaciones como una única gran operación?

Evaluación

- ◆ **Control 12 - Obligatorio** - 2 preguntas sobre la clase de hoy. Se puede responder hasta este viernes a las 20:00.
- ◆ Próxima semana se entrega hito 1 y se publica siguiente enunciado del hito 2 y 3.

IIC2523

Sistemas Distribuidos

— Hernán F. Valdivieso López —
(2026 - 2 / Clase 12)

Créditos (animes utilizados)

Itai no wa Iya nano de Bōgyoryoku ni Kyokufuri Shitai to Omoimasu (Bofuri)



Tensei shitara Slime Datta Ken



Kimetsu no Yaiba



Puella Magi Madoka Magica

